Lab 41: Vague passages forced into a category they don't fit

Topic 4: Prompt Engineering & Structured Output · Task 4.3: Enforce structured output using tool use and JSON schemas

Runs on Simulator 3: Multi-Agent Research System · Self-practice

Source: SkillCertPro practice exam Part 1, Q28 (unofficial third-party question)

Commit to an answer first

The document-analysis subagent extracts an evidence_type field using a JSON Schema enum of experiment , survey , expert_interview , and market_report . In testing, vague passages are often forced into a specific category, while new source types sometimes produce values outside the schema and fail downstream validation. What change best improves the schema design while preserving reliable categorization?

A. Add unclear and other enum values, plus an other_detail field, and instruct when to use each.

B. Remove the enum constraint from evidence_type and ask each subagent to write the best category label.

C. Keep the current enum list and instruct the model to select the closest category for every passage.

D. Make evidence_type required and retry whenever validation fails until a listed category is returned.

Simulate the solution

```
# In the browser: open Simulator 3, then Lab 41
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 41

# In a terminal, from simulators/simulator-3-multi-agent-research/
uv run research_system.py --case UC-28
```

What you should see

Every value stays in the enum. Vague passages use unclear. Any other carries other_detail.

All evidence_type values are inside the enum; vague passages use `unclear`; any `other` carries other_detail.

Open `simulators/simulator-3-multi-agent-research/evidence/` and find it yourself.

The answer

A. Add unclear and other, plus an other_detail field.

A robust schema constrains predictable values while still representing edge cases honestly. An enum that has no valid answer for ambiguous or novel inputs pushes the model toward false precision. unclear gives it a valid way to report insufficient evidence. other plus other_detail preserves information about emerging categories without accepting arbitrary strings in the primary field. This is an application-level schema design pattern, not a special Anthropic feature.

Why the others fail

B. Free-text labels lose categorical consistency and break downstream consumers.

C. Choosing the “closest” category is the false-precision problem itself.

D. Retrying until a listed value comes back forces more false precision, at extra cost.

Check yourself

- every evidence_type is a member of the six-value enum
- no finding is forced into a specific category when the text is vague (at least one unclear or other, or a justification)
- every other has a non-empty other_detail

Now change one thing

In fixtures/sources.json replace the visual-arts TIMEOUT with [] and rerun: a valid empty result must not read as a failure.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.